

CS3212 Algorithms

Aya Zirikly
09/17/2026

Quiz 2

09/24/2026

HeapSort Complexity & Trade-offs —Time Complexity

Metric	Complexity
Best Case	$\Theta(n \log n)$
Average Case	$\Theta(n \log n)$
Worst Case	$\Theta(n \log n)$
Auxiliary Space	$O(1)$
Stability	Not Stable

Stability: NOT Stable

Long-distance swaps between root $A[1]$ and end element $A[i]$ break the relative ordering of duplicate keys.

HeapSort Complexity & Trade-offs —Time Complexity

HeapSort vs. QuickSort in Practice

- **Guaranteed vs. Expected Performance:**
 - **HeapSort:** Hard $\Theta(n \log n)$ worst-case guarantee; no worst-case input triggers quadratic time.
 - **QuickSort:** $O(n \log n)$ average case, but risks worst-case $O(n^2)$ without proper pivot choices.

Breaking the $O(n \log n)$ Barrier

- ❖ Comparison-based sorting algorithms (like QuickSort or MergeSort) must compare two elements at a time ("Is $A > B$?").
 - Because they treat data as "black boxes"—they know nothing about what the values actually represent—they are mathematically bound by an $O(n \log n)$ lower speed limit.
- ❖ **Leveraging input structure:** make specific assumptions about the nature of the data.
 - When you know *what* the data looks like in advance, you can place elements directly into calculated memory locations without comparing them head-to-head, achieving **$O(n)$ linear time**.
 - Utilize specific assumptions about the distribution or type of input data (e.g., fixed range, integer digits, uniform continuous intervals).

Breaking the $\Omega(n \log n)$ Barrier

The Barrier: The decision tree model proves that any comparison-based sort (such as Merge Sort, Quick Sort, or Heap Sort) requires $\Omega(n \log n)$ operations in the worst case.

Why It Holds: A decision tree sorting n elements must have at least $n!$ leaves to represent every possible permutation. The minimum height of such a tree is $\log_2(n!)$, which simplifies asymptotically to $\Omega(n \log n)$.

Why $n!$ Leaves? (Every Possible Outcome)

- ❖ An unsorted array of n unique elements has $n!$ (n factorial) possible sorted arrangements (permutations).
- ❖ For $n = 3$ elements (A, B, C), there are $3! = 6$ possible sorted orders:
 - (A,B,C), (A,C,B), (B,A,C), (B,C,A), (C,A,B), (C,B,A).
- ❖ To sort any possible input correctly, the decision tree must have at least $n!$ leaf nodes (outcomes) at the bottom so that every potential valid ordering is reachable.

Why Does Tree Height Equal Comparisons?

- ❑ Every internal node in the tree represents a single comparison
 - ❑ Is $A[i] \leq A[j]$?
 - ❑ Answering "Yes" branches left
 - ❑ Answering "No" branches right.
- ❑ The longest path from the root down to a leaf represents the tree height (h), which equals the maximum number of comparisons performed in the worst case.

Why Is Height $h \geq \log_2(n!)$? (Binary Tree Capacity)

In a binary tree where every decision splits 2 ways:

- Height 1 holds at most $2^1 = 2$ leaves.
- Height 2 holds at most $2^2 = 4$ leaves.
- Height h holds at most 2^h leaves.

To fit at least $n!$ leaves inside the tree, the height h must satisfy $2^h \geq n!$.

Taking the base-2 logarithm of both sides gives **$h \geq \log_2(n!)$** .

Why Does $\log_2(n!)$ Simplify to $\Omega(n \log n)$?

By definition of factorials: $n! = n * (n - 1) * (n - 2) * \dots * 1$. Taking the logarithm converts multiplication into addition:

$$\log_2(n!) = \log_2(n) + \log_2(n - 1) + \dots + \log_2(1)$$

The upper half of these terms (from n down to $n / 2$) are all at least as large as $\log_2(n / 2)$. Since there are $n / 2$ such terms:

$$\log_2(n!) \geq (n / 2) * \log_2(n / 2) = (n / 2) * (\log_2(n) - 1) = \Omega(n \log n)$$

Because every comparison sort maps to a decision tree, no comparison-based algorithm (MergeSort, HeapSort, QuickSort) can ever beat $\Omega(n \log n)$ time in the worst case.

Key Structural Assumptions & Examples

Fixed Range (Counting Sort)

Assumption: The input consists of integers within a known, small range from 0 to k.

Mechanism: Create an auxiliary array of size k. Use the values of the input items as direct array indices to tally occurrences, reconstruct the sorted array.

Example: Sort 1,000,000 student exam scores between 0 and 100.

- *Without comparison:* Instead of comparing student A's score to student B's, initialize an array of length 101. Iterate through the list once and increment `count[score]`. A score of 85 maps directly to index 85.

Uniform Continuous Intervals (Bucket Sort)

Assumption: Data consists of real numbers distributed evenly over a continuous range, typically the half-open interval $[0, 1)$.

Mechanism: Partition the range into n equal sub-intervals (buckets). Calculate which bucket an element belongs to using direct math: $\text{index} = \text{floor}(n * \text{value})$.

Example: Sort 100,000 sensor readings ranging smoothly from 0.000 to 0.999.

- *Without comparison:* A reading of 0.421 immediately maps to bucket index $\text{floor}(100000 * 0.421) = 42100$. Because the data is uniformly spread out, almost every bucket gets 1 or 2 items, which can be combined instantly.

Bucket-sort

Bucket Sort works by partitioning the input range into equal sub-intervals ("buckets"), scattering data into these buckets using direct mathematical mapping, sorting each bucket individually, and collecting the results in order.

Bucket-sort example

Input Dataset An array A of $n = 10$ real numbers uniformly distributed across the interval $[0, 1)$:

$A = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.68, 0.37]$

Step 1: Initialize Buckets Create $n = 10$ empty buckets corresponding to indices 0 through 9:

Buckets 0–9: [[], [], [], [], [], [], [], [], [], []]

Bucket-sort example

Step 2. Calculate each element's bucket index using the formula: $\text{index} = \text{floor}(n * \text{value})$.

- $\text{floor}(10 * 0.78) = \text{floor}(7.8) = 7 \rightarrow$ place 0.78 in Bucket 7
- $\text{floor}(10 * 0.17) = \text{floor}(1.7) = 1 \rightarrow$ place 0.17 in Bucket 1
- $\text{floor}(10 * 0.39) = \text{floor}(3.9) = 3 \rightarrow$ place 0.39 in Bucket 3
- $\text{floor}(10 * 0.26) = \text{floor}(2.6) = 2 \rightarrow$ place 0.26 in Bucket 2
- $\text{floor}(10 * 0.72) = \text{floor}(7.2) = 7 \rightarrow$ place 0.72 in Bucket 7
- $\text{floor}(10 * 0.94) = \text{floor}(9.4) = 9 \rightarrow$ place 0.94 in Bucket 9
- $\text{floor}(10 * 0.21) = \text{floor}(2.1) = 2 \rightarrow$ place 0.21 in Bucket 2
- $\text{floor}(10 * 0.12) = \text{floor}(1.2) = 1 \rightarrow$ place 0.12 in Bucket 1
- $\text{floor}(10 * 0.68) = \text{floor}(6.8) = 6 \rightarrow$ place 0.68 in Bucket 6
- $\text{floor}(10 * 0.37) = \text{floor}(3.7) = 3 \rightarrow$ place 0.37 in Bucket 3

Bucket-sort example

Buckets state after scattering:

- **Bucket 0:** []
- **Bucket 1:** [0.17, 0.12]
- **Bucket 2:** [0.26, 0.21]
- **Bucket 3:** [0.39, 0.37]
- **Bucket 4:** []
- **Bucket 5:** []
- **Bucket 6:** [0.68]
- **Bucket 7:** [0.78, 0.72]
- **Bucket 8:** []
- **Bucket 9:** [0.94]

Bucket-sort example

Step 3: Sort Individual Buckets Sort each non-empty bucket using Insertion Sort:

- **Bucket 1:** $[0.17, 0.12]$ becomes $[0.12, 0.17]$
- **Bucket 2:** $[0.26, 0.21]$ becomes $[0.21, 0.26]$
- **Bucket 3:** $[0.39, 0.37]$ becomes $[0.37, 0.39]$
- **Bucket 6:** $[0.68]$ remains $[0.68]$
- **Bucket 7:** $[0.78, 0.72]$ becomes $[0.72, 0.78]$
- **Bucket 9:** $[0.94]$ remains $[0.94]$

Bucket-sort example

Step 4: Gather Sorted Values Iterate through buckets 0 to 9 in sequential order and concatenate all elements back into the main array:

- Sorted Array = [0.12, 0.17, 0.21, 0.26, 0.37, 0.39, 0.68, 0.72, 0.78, 0.94]

Bucket sort -worst case

The Three-Phase Pipeline



1. Scatter Phase

Input elements are partitioned into k equal-interval buckets using a deterministic hash mapping function $f(x)$.



2. Sub-Sort Phase

Each non-empty bucket is sorted independently using a secondary sorting algorithm



3. Gather Phase

The individually sorted buckets are traversed sequentially in order, concatenating elements into the final sorted array.

Uniform vs. Skewed Distribution

Ideal Case: Uniformity

When input elements are uniformly distributed across the range $[0, 1)$, elements scatter evenly across all k buckets (approximately n/k items per bucket).

Sorting tiny buckets yields linear expected performance: $O(n + k)$.

Worst Case: Severe Skew

When data suffers from high clustering, all n elements land inside a single bucket while remaining $k - 1$ buckets stay empty.

The scatter benefit collapses entirely into quadratic time: $O(n^2)$.

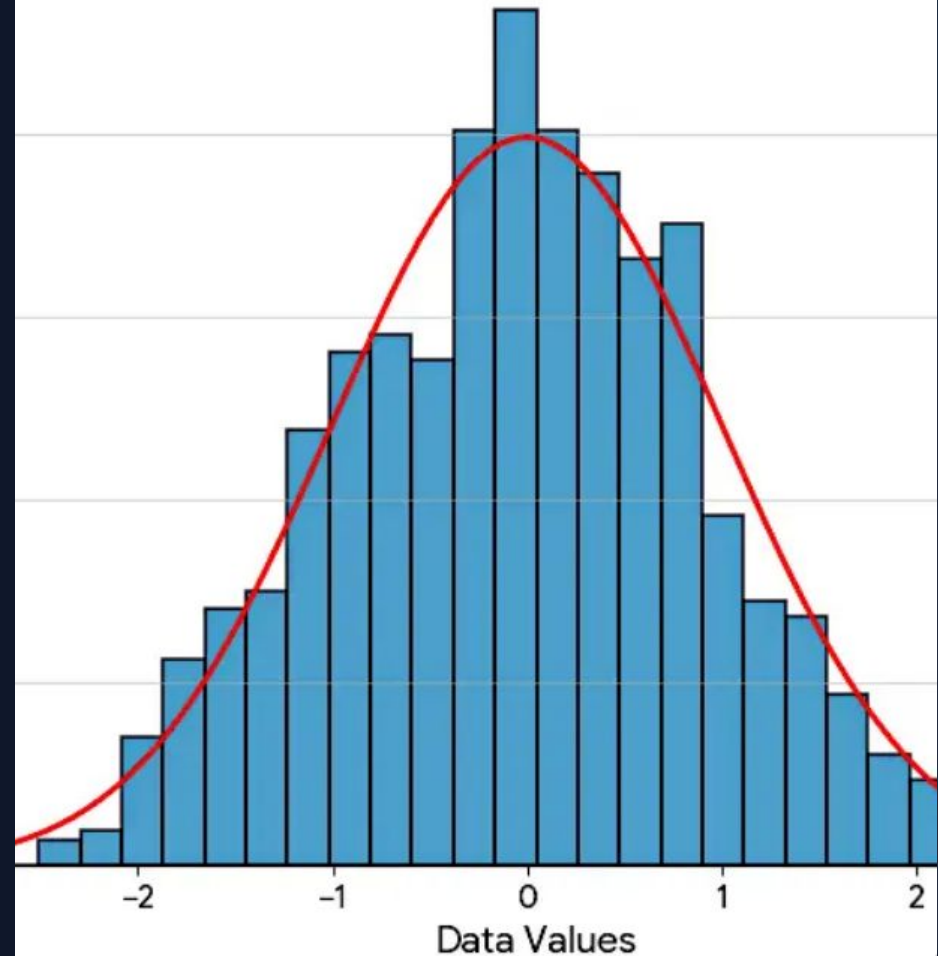
What Causes the Worst Case?

Data Clustering & Mapped Collisions

The worst case is triggered when key values fall within an extremely narrow numerical interval relative to bucket boundaries.

If the hash function maps every key to the same bucket index, the divide-and-conquer optimization completely fails, reducing the algorithm to an un-partitioned list.

Normal Distribution Histogram ($\mu = 0, \sigma =$



Peak Runtime Degradation

$O(n^2)$

Worst-Case Time Complexity

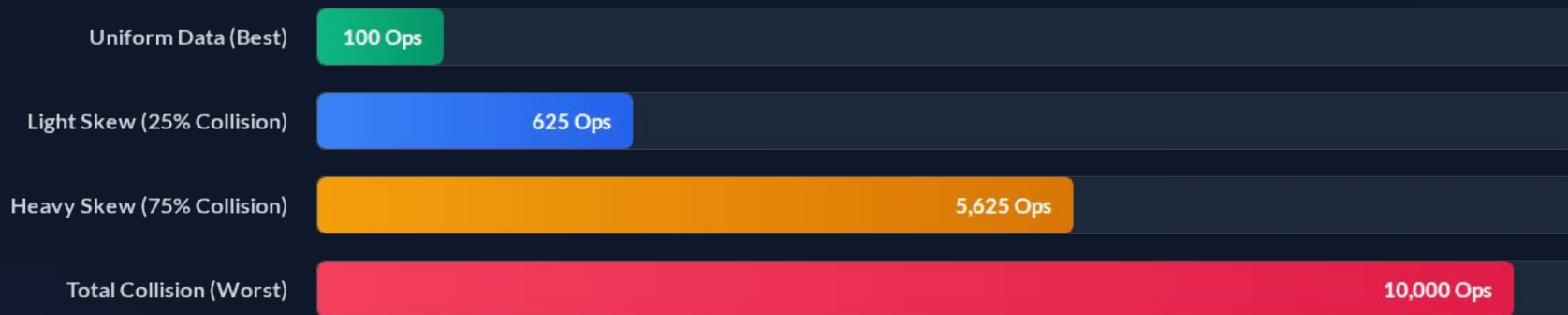
Mathematical Derivation

Suppose all n elements fall into bucket $B[0]$:

$$T(n) = O_{\text{scatter}}(n) + O_{\text{sort}}(n^2) + O_{\text{gather}}(n)$$

With Insertion Sort running on a single bucket of size n , total comparisons reach $n(n - 1) / 2$, leading directly to quadratic growth.

Growth Rate Under Clustering



Computational operations scale quadratically as clustering shifts elements from multi-bucket distribution to single-bucket concentration ($N = 100$).

Algorithm Worst Case Comparison

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$

Space Complexity Overhead

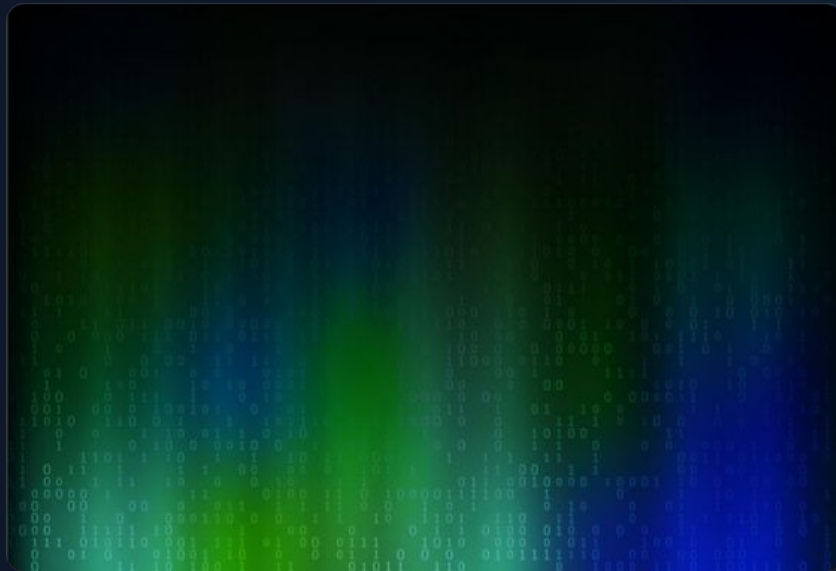
-  **Auxiliary Space Requirement $O(n + k)$:** Bucket Sort demands extra memory for k bucket headers and n element pointer references, making it non-in-place.
-  **Wasted Allocation Overhead:** In the worst-case scenario, $k - 1$ bucket structures sit entirely empty while still consuming allocation overhead.
-  **Dynamic Memory Resizing:** Dynamic bucket lists incur frequent memory reallocation overhead (grow-and-copy) when elements pile uncontrollably into a single container.

Architectural Mitigations

🛡️ **$O(n \log n)$ Sub-Sorters:** Replace Insertion Sort with Merge Sort inside buckets to cap worst-case time at $O(n \log n)$.

📈 **Adaptive Bucket Boundaries:** Estimate data distribution via CDF sampling to dynamic-size buckets for non-uniform data.

🔗 **Recursive Bucket Splitting:** Automatically re-partition any bucket whose population exceeds threshold τ .



Core Takeaway

"Bucket sort's high efficiency is not an intrinsic guarantee of the code, but a structural contract with the underlying data distribution."

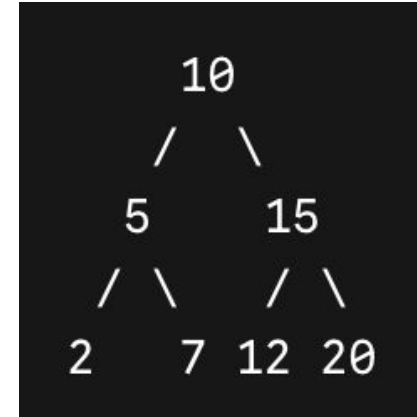
– Principles of Algorithm Design

Binary Search Tree (BST) Definition & Properties

Definition: A node-based binary tree data structure where each node contains a key, a value, and references to at most two child nodes (**left** and **right**).

The BST Invariant (Property): For any given node **X**:

- Every key in **X.left** subtree is strictly less than **X.key**.
- Every key in **X.right** subtree is strictly greater than **X.key**.
- Both **left** and **right** subtrees must also be valid BSTs.



Lookup (Search) Operation

Compare target key against the current node.

If target is smaller, move to left child;

if larger, move to right child.

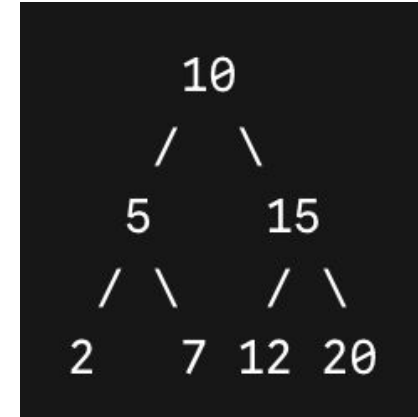
Repeat until match is found or a **NULL** pointer is reached.

```
SEARCH(node, target):  
    if node is NULL or node.key == target:  
        return node  
    if target < node.key:  
        return SEARCH(node.left, target)  
    else:  
        return SEARCH(node.right, target)
```

Example Trace

Search for key 12:

- Start at root 10: $12 > 10$ -> Move Right to 15
- At node 15: $12 < 15$ -> Move Left to 12
- At node 12: $12 == 12$ -> Return Node 12 (Success)



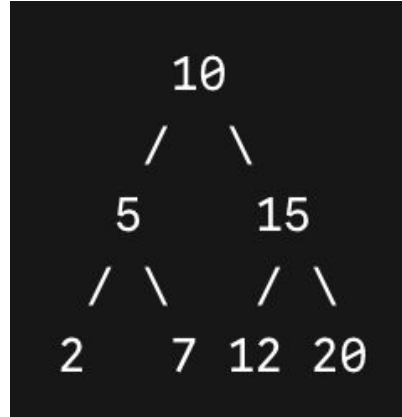
Insertion Operation

- ❑ Search for target key until reaching a **NULL** position, then attach the new node as a leaf at that spot.
- ❑ Duplicate keys are typically disallowed or routed consistently to one side.

```
INSERT(root, key):  
    if root is NULL:  
        return NEW_NODE(key)  
    if key < root.key:  
        root.left = INSERT(root.left, key)  
    else if key > root.key:  
        root.right = INSERT(root.right, key)  
    return root
```

Example Trace

Insert key 8 into current tree



Example Trace

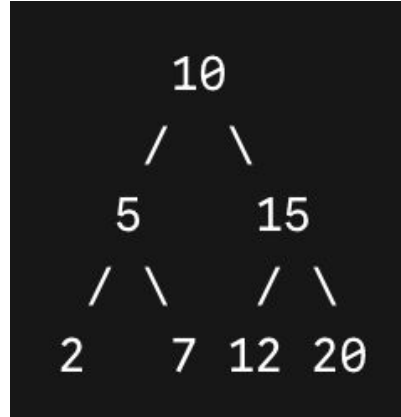
Insert key 8 into current tree

Compare to 10: $8 < 10$ -> Go Left to 5

Compare to 5: $8 > 5$ -> Go Right to 7

Compare to 7: $8 > 7$ -> Go Right -> Reaches NULL

Create new node 8 and set $7.\text{right} = \text{Node}(8)$



Deletion

```
DELETE(root, key):
    if root is NULL:
        return NULL
    if key < root.key:
        root.left = DELETE(root.left, key)
    else if key > root.key:
        root.right = DELETE(root.right, key)
    else:
        // Case 1 & 2: 0 or 1 child
        if root.left is NULL:
            return root.right
        else if root.right is NULL:
            return root.left

        // Case 3: 2 children
        successor = FIND_MIN(root.right)
        root.key = successor.key
        root.right = DELETE(root.right, successor.key)
    return root

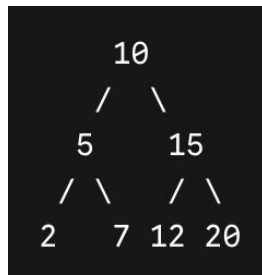
FIND_MIN(node):
    while node.left is not NULL:
        node = node.left
    return node
```

Example trace

Delete Root 10 from tree with children 5 and 15 (case 3)

Node 10 has 2 children -> Find minimum in right subtree (15's subtree) → Successor is 12.

Copy 12 into root position: Root key becomes 12.



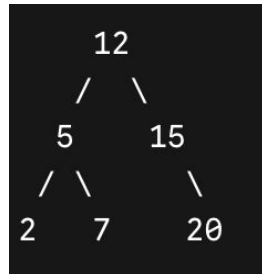
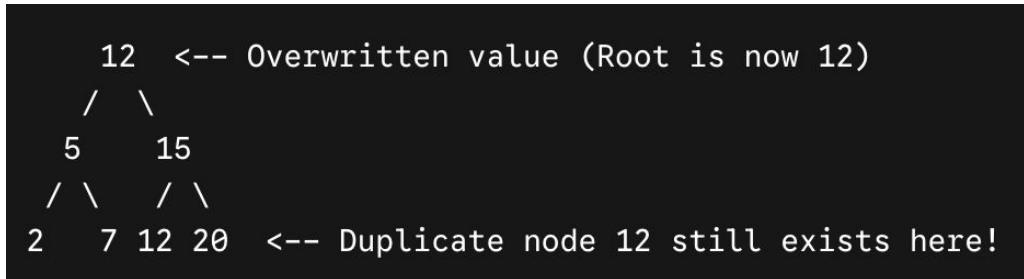
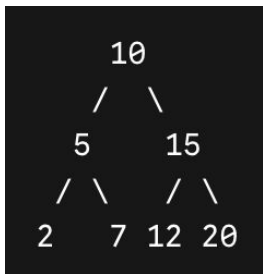
```
    12  <-- Overwritten value (Root is now 12)
   /  \
  5    15
 /  \  /  \
2   7 12 20  <-- Duplicate node 12 still exists here!
```

Example trace

The tree temporarily contains two nodes with key 12.

Remove the original node 12 sitting in the right subtree (`root.right → 15`).

Call `DELETE(root.right, 12)` => "Look inside the right subtree starting at node 15, find key 12, and delete that node."

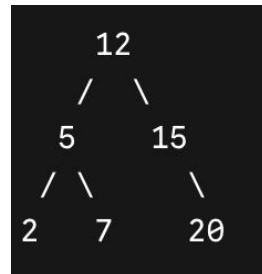
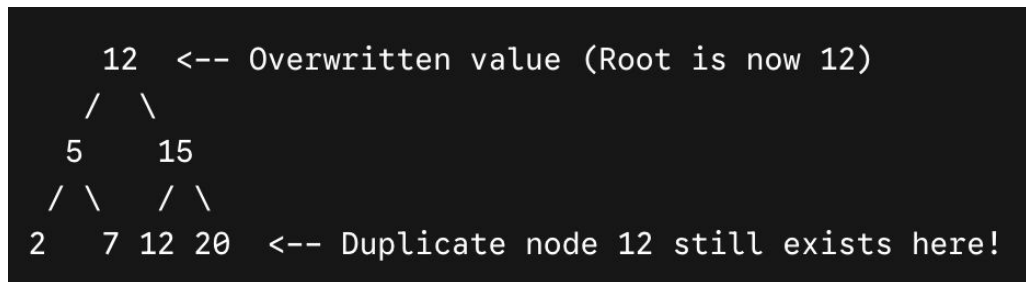
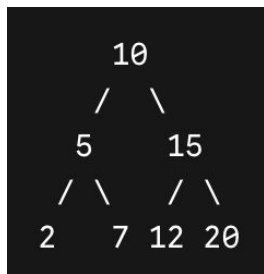


Example trace

The tree temporarily contains two nodes with key 12.

Remove the original node 12 sitting in the right subtree (`root.right → 15`).

Call `DELETE(root.right, 12)` => "Look inside the right subtree starting at node 15, find key 12, and delete that node."



Why is this always Case 1 or Case 2?

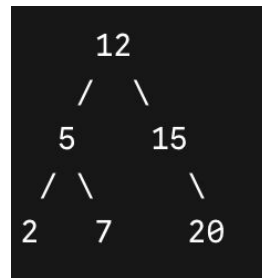
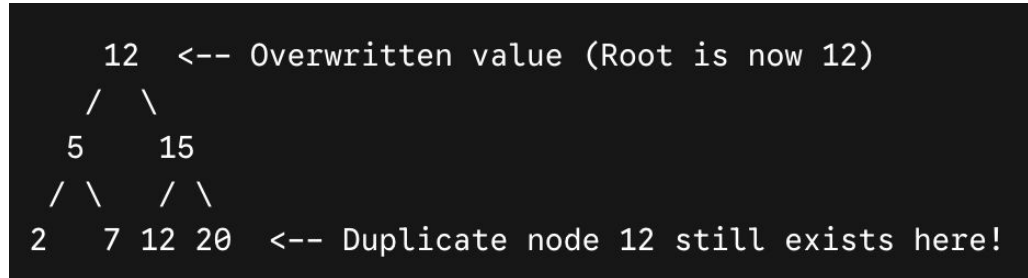
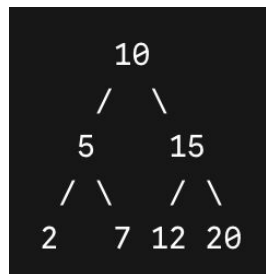
Because 12 was chosen as the **minimum** node in the right subtree

- It **cannot have a left child**. (If it had a left child, that left child would be smaller than 12, so 12 wouldn't have been the minimum).

Since the node to delete has no left child, it can only ever be:

- **Case 1:** A leaf node with 0 children.
- **Case 2:** A node with 1 child (only a right child).

In this specific example, node 12 has no children at all (Case 1). The node 15 simply sets `15.left = NULL`.



Time & Space Complexity Analysis

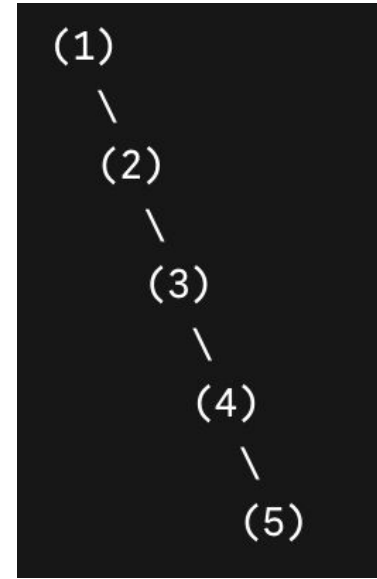
Height Dependency: All basic BST operations (Search, Insert, Delete) run in $O(h)$ time, where h is the height of the tree.

In a degenerate tree (binary tree where every parent node has only one child) $h = n - 1$, instead of the optimal $h = \log_2(n)$

Search $O(n)$

Searching in a balanced tree eliminates half the remaining nodes at each step. In a degenerate tree, every node has only one child, forcing the search path to visit every single node sequentially from root to leaf, behaving like a linked list.

- **Example:** Searching for 5 in the skewed tree above:
 - Compare to 1: $5 > 1$ -> go right to 2
 - Compare to 2: $5 > 2$ -> go right to 3
 - Compare to 3: $5 > 3$ -> go right to 4
 - Compare to 4: $5 > 4$ -> go right to 5
 - Match found at node 5.
 - **Result:** Inspected all $n = 5$ nodes.



Insertion $O(n)$

Insertion requires locating the correct **NULL** position before attaching the new node. In a skewed tree, finding that spot requires walking down the entire height of n nodes.

- **Example:** Inserting 6 into the skewed tree above:
 - Traverse 1 -> 2 -> 3 -> 4 -> 5 to confirm 6 belongs on the right of 5.
 - Set 5.right = Node(6).
 - **Result:** $n = 5$ traversal steps before completing the insertion.

Deletion $O(n)$

Deletion requires first searching for the node to remove, then adjusting pointers. If the node to delete is near the bottom or at the deepest leaf, locating it takes $O(n)$ steps.

- ❑ **Example:** Deleting 5 from the skewed tree above:
 - ❑ Traverse 1 -> 2 -> 3 -> 4 to reach node 5.
 - ❑ Identify 5 as a leaf node (0 children) and set 4.right = NULL.
 - ❑ **Result:** Traversal phase takes $O(n)$ time, dominating the constant pointer modification.

Space Complexity $O(n)$ Auxiliary Stack Space

Storing n elements always takes $O(n)$ structural memory. However, recursive implementations of Search, Insert, and Delete consume call stack memory proportional to tree height h .

- ❑ **Example:** In a balanced tree of 1,000,000 nodes, tree height is ~ 20 , requiring only 20 stack frames.
 - ❑ In a worst-case skewed tree of 1,000,000 nodes, recursive calls pile up to 1,000,000 levels deep, leading to $O(n)$ stack memory usage and risks of stack overflow errors.

To prevent binary search trees from degenerating into linear paths, self-balancing trees (such as AVL trees or Red-Black trees)

Metric	Perfectly Balanced (Best Case)	Randomly Built (Average Case)	Sorted Input (Worst Case)
Tree Shape	Completely symmetric	Reasonably spread out	Single linear branch
Height (h)	$\log_2(n)$	$\sim 1.39 * \log_2(n)$	$n - 1$
Search / Insert / Delete	$O(\log n)$	$O(\log n)$	$O(n)$

Summary

Operation	Best Case (Balanced)	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$
Space Complexity	$O(n)$ total space	$O(n)$ total space	$O(n)$ total space ($O(n)$ call stack)

Summary

Operation	Best Case (Balanced)	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(l)$	Memory consumed by recursive calls Every time a recursive function calls itself, the computer pushes a "stack frame" into execution memory to track the function's variables and return address.
Deletion	$O(\log n)$	$O(l)$	
Space Complexity	$O(n)$ total space	$O(n)$ total space	$O(n)$ total space ($O(n)$ call stack)